

Taller 3 IA

Integrantes:

Ivonne Cuncanchun – 202420383

Carla González - 202411176

Samantha Puentes - 202410295

Sebastián Martínez - 202415764

Punto 1: Model Checking

Pruebas:

La implementación del módulo de Model Checking fue validada mediante la ejecución de las pruebas unitarias del proyecto (python -m pytest -v). Se verificó que las funciones generaran correctamente todos los modelos posibles (get_all_models), identificaran fórmulas satisfacibles e insatisfacibles (check_satisfiable), reconocieran tautologías (check_valid), evaluaran correctamente la consecuencia lógica entre una base de conocimiento y una consulta (check_ entailment) y construyeran adecuadamente las tablas de verdad (truth_table). Durante las pruebas se detectó un caso especial en check_ entailment cuando la base de conocimiento era vacía; este se corrigió verificando directamente la validez de la consulta, ya que una base de conocimiento vacía únicamente implica tautologías. Tras esta corrección, todas las pruebas unitarias del módulo fueron superadas satisfactoriamente.

```
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckSatisfiable::test_contradiction_unsatisfiable PASSED [ 59%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckSatisfiable::test_disjunction_satisfiable PASSED [ 60%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckSatisfiable::test_complex_satisfiable PASSED [ 62%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckValid::test_tautology PASSED [ 63%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckValid::test_not_tautology PASSED [ 64%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckValid::test_implication_tautology PASSED [ 65%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckValid::test_double_negation_tautology PASSED [ 67%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckValid::test_modus_ponens_tautology PASSED [ 68%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckEntailment::test_simple_entailment PASSED [ 69%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckEntailment::test_no_entailment PASSED [ 70%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckEntailment::test_contradiction_entails_everything PASSED [ 71%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckEntailment::test_chain_entailment PASSED [ 73%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestCheckEntailment::test_empty_kb PASSED [ 74%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestTruthTable::test_simple_atom PASSED [ 75%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestTruthTable::test_conjunction PASSED [ 76%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestTruthTable::test_implication PASSED [ 78%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestTruthTable::test_three_atoms PASSED [ 79%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestTruthTable::test_biconditional PASSED [ 80%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexSatisfiable::test_four_atom_satisfiable PASSED [ 81%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexSatisfiable::test_xor_satisfiable PASSED [ 82%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexSatisfiable::test_xor_has_two_models PASSED [ 84%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexValid::test_de_morgan_tautology PASSED [ 85%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexValid::test_hypothetical_syllogism PASSED [ 86%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexValid::test_disjunctive_syllogism PASSED [ 87%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexValid::test_not_tautology_four_atoms PASSED [ 89%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexEntailment::test_modus_tollens PASSED [ 90%]
T3_WorldCupGroupStage/tests/test_model_checking.py::TestComplexEntailment::test_resolution_chain PASSED [ 91%]
```

Punto 2: Conversión a CNF

Resultados/ Pruebas:

La implementación de la conversión a CNF fue validada con los test respectivos. Cada una de las transformaciones (eliminate_iff, eliminate_implication, push_negation_inward, distribute_or_over_and y flatten), lograron un funcionamiento correcto. Se superaron las 42 pruebas.

```
PS C:\Users\ivonn\Downloads\Taller-3-IA\T3_WorldCupGroupStage\tests> python -m pytest test_cnf.py -v
test_cnf.py::TestEliminateIff::test_nested_iff PASSED [ 4%]
test_cnf.py::TestEliminateIff::test_no_iff PASSED [ 7%]
test_cnf.py::TestEliminateIff::test_iff_inside_not PASSED [ 9%]
test_cnf.py::TestEliminateIff::test_iff_inside_and PASSED [ 11%]
test_cnf.py::TestEliminateImplication::test_simple_implication PASSED [ 14%]
test_cnf.py::TestEliminateImplication::test_nested_implication PASSED [ 16%]
test_cnf.py::TestEliminateImplication::test_no_implication PASSED [ 19%]
test_cnf.py::TestEliminateImplication::test_implication_in_and PASSED [ 21%]
test_cnf.py::TestPushNegationInward::test_de_morgan_and PASSED [ 23%]
test_cnf.py::TestPushNegationInward::test_de_morgan_or PASSED [ 26%]
test_cnf.py::TestPushNegationInward::test_no_negation PASSED [ 28%]
test_cnf.py::TestPushNegationInward::test_nested_de_morgan PASSED [ 30%]
test_cnf.py::TestPushNegationInward::test_deeply_nested PASSED [ 33%]
test_cnf.py::TestPushNegationInward::test_nary_de_morgan PASSED [ 35%]
test_cnf.py::TestDistributeOrOverAnd::test_simple_distribution PASSED [ 38%]
test_cnf.py::TestDistributeOrOverAnd::test_no_distribution_needed PASSED [ 40%]
test_cnf.py::TestDistributeOrOverAnd::test_and_stays PASSED [ 42%]
test_cnf.py::TestDistributeOrOverAnd::test_nested_distribution PASSED [ 45%]
test_cnf.py::TestDistributeOrOverAnd::test_both_sides_and PASSED [ 47%]
test_cnf.py::TestDistributeOrOverAnd::test_or_with_multiple_and_children PASSED [ 50%]
test_cnf.py::TestFlatten::test_flatten_and PASSED [ 52%]
test_cnf.py::TestFlatten::test_flatten_or PASSED [ 54%]
test_cnf.py::TestFlatten::test_no_flatten_needed PASSED [ 57%]
test_cnf.py::TestFlatten::test_deep_nesting PASSED [ 59%]
test_cnf.py::TestFlatten::test_mixed_nesting PASSED [ 61%]
test_cnf.py::TestFlatten::test_single_element_after_flatten PASSED [ 64%]
test_cnf.py::TestFlatten::test_flatten_not PASSED [ 66%]
test_cnf.py::TestEliminateDoubleNegation::test_double_negation PASSED [ 69%]
test_cnf.py::TestEliminateDoubleNegation::test_triple_negation PASSED [ 71%]
test_cnf.py::TestEliminateDoubleNegation::test_no_double_negation PASSED [ 73%]
test_cnf.py::TestEliminateDoubleNegation::test_nested_double_negation PASSED [ 76%]
test_cnf.py::TestToCNF::test_simple_implication PASSED [ 78%]
test_cnf.py::TestToCNF::test_distribute_or_over_and PASSED [ 80%]
test_cnf.py::TestToCNF::test_de_morgan PASSED [ 83%]
test_cnf.py::TestToCNF::test_complex_formula PASSED [ 85%]
test_cnf.py::TestToCNF::test_biconditional PASSED [ 88%]
test_cnf.py::TestToCNF::test_complex_nested PASSED [ 90%]
test_cnf.py::TestToCNF::test_nested_biconditional_chain PASSED [ 92%]
test_cnf.py::TestToCNF::test_four_variables_complex PASSED [ 95%]
test_cnf.py::TestToCNF::test_double_negation_inside_iff PASSED [ 97%]
test_cnf.py::TestToCNF::test_or_of_three_conjunctions PASSED [100%]

===== 42 passed in 0.18s =====
```

Punto 3a: Implementación de predicados

Pruebas:

La implementación se verificó mediante la ejecución de las pruebas unitarias del proyecto utilizando python -m pytest -v. Las pruebas correspondientes al punto 3a (test_predicates.py) fueron superadas satisfactoriamente, incluyendo los casos de cálculo de puntos, diferencia de gol, ranking (teams_above y teams_below), clasificación matemática (definitely_qualified) y eliminación matemática (definitely_eliminated).

```
T3_WorldCupGroupStage/tests/test_predicates.py::TestPointsAndGD::test_win_draw_loss_points_and_gd PASSED [ 92%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestPointsAndGD::test_three_wins PASSED [ 93%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestTeamsBelowAbove::test_same_points_better_gd_ranks_higher PASSED [ 95%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestTeamsBelowAbove::test_clear_leader PASSED [ 96%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestDefinitelyQualified::test_mexico_style_clinch PASSED [ 97%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestDefinitelyQualified::test_finished_group_gd_tiebreak_qualifies PASSED [ 98%]
T3_WorldCupGroupStage/tests/test_predicates.py::TestDefinitelyEliminated::test_last_place_eliminated_with_pending PASSED [100%]
```

Punto 3b: Preguntas de análisis cualitativo:

- ¿Qué significa que una conclusión se derive con `holds_in_all`? ¿En qué se parece y en qué se diferencia de $KB \models \phi$ en model checking proposicional?

Una conclusión se deriva con `holds_in_all` cuando una condición es verdadera en todos los escenarios posibles que genera el wngine a partir de los partidos pendientes. Si existe al menos un escenario donde la condición no se cumple, entonces la conclusión no puede asegurarse.

Esto se parece al model checking proposicional porque ambos verifican que una afirmación sea verdadera en todos los modelos posibles. La diferencia acá es que, en el model checking proposicional, los modelos corresponden a todas las asignaciones posibles de valores de verdad para los átomos de una proposición, mientras que con `holds_in_all` los modelos son distintos escenarios deportivos, generados por los posibles resultados de los partidos restantes.

- ¿Qué es fijo y qué varía entre los escenarios de mundos posibles? ¿Qué se perdería si solo usáramos constantes sin cuantificar sobre escenarios?

Lo que permanece fijo es la información ya conocida que son los equipos, los grupos y los resultados de los partidos que ya se disputaron. Lo que varía son solamente los resultados de los partidos pendientes, ya que cada uno puede producir distintos escenarios.

Si solo se utilizara un estado fijo, no se podrían analizar todas las posibilidades futuras. Por lo tanto, no se podría determinar si un equipo está matemáticamente clasificado o eliminado, porque esas conclusiones dependen de que la condición se mantenga en todos los escenarios posibles.

- ¿Por qué los partidos pendientes usan marcadores 1-0 / 0-0 / 0-1? ¿Qué se gana y qué se pierde con esa simplificación?

Se utilizan únicamente los marcadores 1-0, 0-0 y 0-1 porque representan los tres resultados posibles de un partido: victoria local, empate y victoria visitante. Esto reduce de manera significativa el número de escenarios que deben evaluarse. De esta manera se gana eficiencia, ya que el número de mundos posibles es mucho menor. Sin embargo, se pierde precisión en aspectos como la diferencia exacta de gol, pues un 4-0 y un 1-0 producen la misma victoria, aunque afecten de manera diferente el desempate por diferencia de goles.

- ¿Por qué a menudo no se puede determinar si un equipo clasifica hasta la jornada 3 aunque la tabla ya sugiera un orden? ¿Qué condición es necesaria para poder asegurar clasificación?

Aunque la tabla muestre un orden provisional, todavía pueden disputarse partidos cuyos resultados cambien la cantidad de puntos y la diferencia de gol de los equipos. Por ello, la posición actual no garantiza la clasificación. Para asegurar que un equipo clasifica, debe cumplirse que, en todos los escenarios posibles de los partidos pendientes, el equipo permanezca dentro de las posiciones de clasificación. Es decir, debe estar clasificado independientemente de cómo terminen los encuentros restantes.

- Nótese que este taller no modela la regla de los mejores terceros. ¿Qué haría falta para modelarlos con el mismo estilo de escenarios?
Sería necesario generar escenarios para todos los grupos del torneo y no únicamente para un grupo individual. Luego habría que comparar a todos los equipos que terminaran terceros utilizando los criterios oficiales de desempate (puntos, diferencia de gol, goles a favor, etc.) para determinar cuáles clasifican. Esto aumentaría considerablemente la cantidad de escenarios que el sistema tendría que evaluar.

Reflexión: Para cada uno de los puntos anteriores, expliquen brevemente: ¿Qué enfoque utilizaron para su implementación? ¿Qué dificultades encontraron? ¿Cómo verificaron que su implementación era correcta?

Punto 1:

Para implementar el motor de Model Checking se siguió una estrategia donde primero se implementó `get_all_models`, generando todas las asignaciones posibles de valores de verdad para los átomos proposicionales mediante su representación binaria. Luego, las funciones `check_satisfiable`, `check_valid` y `truth_table` se implementaron evaluando la fórmula en cada uno de estos modelos utilizando la función `evaluate`. Por último, `check_ entailment` se implementó aplicando la equivalencia lógica $KB \models q \iff KB \wedge \neg q$ es insatisfacible, verificando la satisfacibilidad de la conjunción entre la base de conocimiento y la negación de la consulta (refutación).

La principal dificultad fue manejar correctamente los casos especiales durante la verificación de la consecuencia lógica. Con los test nos dimos cuenta de que estaba mal implementada la función de `check_ entailment`, se añadió un caso especial que verifica directamente si la consulta es una tautología mediante `check_valid`, ya que una base de conocimiento vacía siempre implica proposiciones válidas.

Se verificó el funcionamiento de la implementación usando el `python -m pytest -v`. Todas las pruebas fueron aprobadas.

Punto 2:

Para implementar la conversión a Forma Normal Conjuntiva (CNF), primero se desarrollaron las funciones `eliminate_iff` y `eliminate_implication`, encargadas de eliminar los bicondicionales e implicaciones de las fórmulas proposicionales. Luego se construyeron `push_negation_inward` y `eliminate_double_negation`, encargadas de desplazar las negaciones hacia los átomos mediante las Leyes de De Morgan y simplificar negaciones dobles. Por último, se aplicó `distribute_or_over_and` para distribuir las disyunciones sobre las conjunciones, y `flatten` para normalizar y aplanar la estructura del árbol de sintaxis abstracta (AST).

La principal dificultad fue manejar las restricciones de aridad del AST en `ast.py` (donde `And` y `Or` exigen al menos dos operandos), lo que obligó a validar la cantidad de hijos resultantes antes de instanciar nuevos nodos. También se tuvo que corregir el recorrido recursivo en `flatten` para asegurar que descendiera correctamente dentro del operador `Not` y controlar la complejidad en la fase de distribución.

Se verificó el funcionamiento de la implementación usando el `python -m pytest -v`. Todas las pruebas fueron aprobadas.

Punto 3:

Para implementar los predicados, primero se desarrollaron las funciones básicas `points` y `goal_difference`, encargadas de calcular los puntos y la diferencia de gol de cada equipo en un escenario determinado. Luego se construyeron `teams_above` y `teams_below`, comparando a cada equipo con sus rivales utilizando como criterios de ordenamiento los puntos y, en caso de empate, la diferencia de gol. Por último, para `definitely_qualified` y `definitely_eliminated` se utilizó el método `holds_in_all` encontrado en el engine.

La principal dificultad fue comprender el funcionamiento del engine de escenarios y cómo utilizar correctamente `holds_in_all` para evaluar una condición en todos los escenarios posibles. También se tuvo que aprender a interpretar adecuadamente las reglas del ranking, asegurando que las comparaciones entre equipos se realizaran únicamente con los criterios establecidos en el enunciado que son los puntos y la diferencia de gol.

Se verificó el funcionamiento de la implementación usando el `python -m pytest -v`. Todas las pruebas fueron aprobadas.

Reflexión uso de IA:

A lo largo del desarrollo de este taller el uso de inteligencia artificial como apoyo fue de gran ayuda en el sentido de comprender algunos conceptos de lógica proposicional que no estaban muy claros, como también revisar la documentación y pequeñas fallas dentro de las funciones del proyecto y del mismo modo resolver dudas puntuales sobre la implementación. La IA también nos fue útil para sugerirnos estrategias de debugging cuando algunas pruebas no pasaban y

para verificar que el enfoque utilizado fuera coherente con lo que se pedía en el taller. En cuanto a la implementación final, la corrección de los errores encontrados y la verificación mediante las pruebas fueron realizadas por nosotros utilizando `python -m pytest -v` para confirmar que el resultado era el esperado. De esta manera, la IA se empleó como una herramienta de apoyo y más no como un agente que solo hace lo que se le pide.